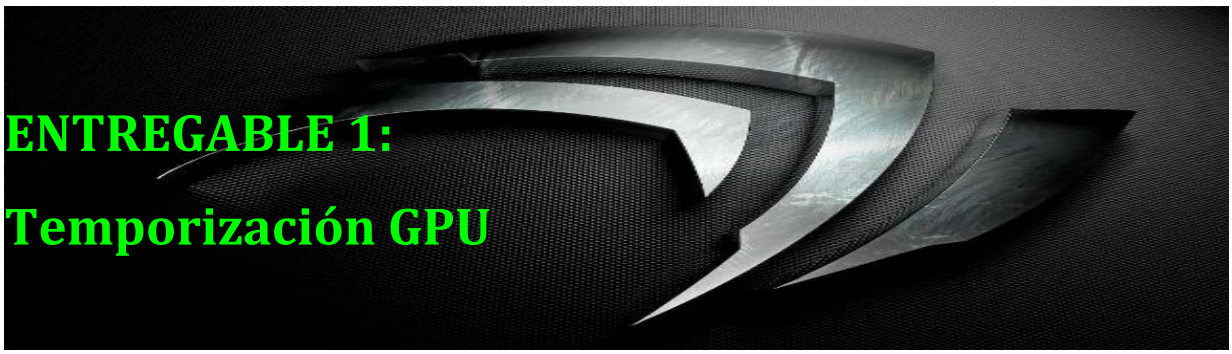


1. Objetivo

Medir el tiempo de ejecución de un *kernel*.

2. Ejercicio

Dada una matriz rectangular con *F* filas y *C* columnas introducidas por el usuario, lanzar un *kernel* bidimensional de un solo bloque que genere una nueva matriz intercambiando filas pares e impares. La matriz inicial se debe inicializar en el *host* de modo que cada fila contenga un mismo número aleatorio entre 1 y 9.

El programa debe mostrar las propiedades del dispositivo seleccionado, el número de hilos lanzados, el tiempo de ejecución del *kernel* y las matrices inicial y final.

```
C:\ProgramData\NVIDIA Corporation\CUDA Samples\v8.0\_ARCO\ARCO\../../bin/win64/Debug/practica.exe
*****************************************************
DEVICE 0: NVIDIA GeForce GT 730
*****************************************************
> CUDA Toolkit                          : 8.0
> Capacidad de Computo                  : 3.5
> Arquitectura CUDA                     : KEPLER
> No. de MultiProcesadores              : 2
> No. de CUDA Cores (192x2)             : 384
> No. maximo de Hilos (por bloque)      : 1024
> Memoria Global (total)                : 2048 MiB
*****************************************************
  Introduce el numero de filas: 5
  Introduce el numero de columnas: 16
> Matriz de 5 FILAS x 16 COLUMNAS
> KERNEL de 1 BLOQUE con 80 HILOS (16x5)
> Tiempo de ejecucion: 0.062496 ms
> MATRIZ ORIGINAL:
 6  6  6  6  6  6  6  6  6  6  6  6  6  6  6  6
 9  9  9  9  9  9  9  9  9  9  9  9  9  9  9  9
 8  8  8  8  8  8  8  8  8  8  8  8  8  8  8  8
 5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5
 9  9  9  9  9  9  9  9  9  9  9  9  9  9  9  9

> MATRIZ FINAL:
 9  9  9  9  9  9  9  9  9  9  9  9  9  9  9  9
 6  6  6  6  6  6  6  6  6  6  6  6  6  6  6  6
 5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5
 8  8  8  8  8  8  8  8  8  8  8  8  8  8  8  8
 9  9  9  9  9  9  9  9  9  9  9  9  9  9  9  9
```

Figura 1.1.: Salida por pantalla de una posible solución.

3. Elementos CUDA

Para realizar esta práctica se pueden utilizar las siguientes funciones:

- **cudaEventCreate** (cudaEvent_t *marca*)
  Devuelve en la varible *marca* un evento para almacenar marcas de tiempo.
- **cudaEventRecord**(cudaEvent_t *marca*, cudaStream_t *stream*)
  Graba una marca de tiempo en el evento *marca*. El valor *stream* especifica la lista de tareas a la que pertenece el evento, que por defecto es 0.
- **cudaEventSynchronize**(cudaEvent_t *marca*)
  Detiene la ejecución del *host* hasta que el *device* haya alcanzado el evento señalado como *marca*.
- **cudaEventDestroy**(cudaEvent_t *marca*)
  Elimina el evento *marca*.
- **cudaEventElapsedTime**(float *tiempo*, cudaEvent_t *marca1*, cudaEvent_t *marca2*)
  Almacena en la varible *tiempo* la diferencia de tiempo en milisegundos transcurridos entre los eventos indicados como *marca1* y *marca2*.

4. Conceptos teóricos

4.1. Sincronismo

En nuestro modelo de programación heterogéneo disponemos de dos elementos (el *host* y el *device*) que trabajan de forma independiente. Esto quiere decir que los dos elementos pueden estar trabajando simultáneamente. Esta forma de operación se conoce como *asíncrona*. Así, cuando lanzamos un *kernel* la GPU comienza a ejecutar su correspondiente código y la CPU continua ejecutando el suyo (el programa principal *main*) sin esperar a que la GPU haya terminado. Esto es muy interesante desde el punto de vista del rendimiento porque permite aprovechar al máximo todos los recursos de computación pero, por otro lado, complica la medida del tiempo de ejecución ya que cualquier temporizador que utilicemos en el programa principal (*main*) sólo nos va a medir el tiempo de CPU.

Esta característica asíncrona de nuestro sistema hace que para de medir el tiempo de ejecución de una aplicación CUDA tengamos que utilizar unas marcas de tiempo en la GPU denominadas *eventos*, para después leerlas desde el programa principal y así poder medir la diferencia de tiempo entre las marcas. Como veremos a continuación, esto implica además tener que sincronizar ambos dispositivos, es decir, detener la ejecución del programa

principal hasta estar seguros de que el trabajo realizado por la GPU se ha completado antes de leer las marcas.

4.2. Eventos (*Events*)

Un evento o *event* en CUDA es esencialmente una marca de tiempo en la GPU que nosotros podemos grabar en un determinado instante de tiempo. La forma de medir el tiempo de ejecución es relativamente fácil, ya que básicamente consiste en crear *eventos* para después grabar en ellos marcas temporales con el fin de calcular la diferencia de tiempos entre las marcas. Los eventos se almacenan en un nuevo tipo de dato denominado **`cudaEvent_t`**, y la función para crear un evento es **`cudaEventCreate()`**:

```
cudaEventCreate (cudaEvent_t *marca);
```

Debemos crear tantos eventos como marcas temporales vayamos a necesitar, que en general serán dos, una de inicio y otra de fin. Una vez creados los eventos donde vamos a guardar las marcas inicial y final, podremos grabar dichas marcas temporales en el momento que deseemos. Esta grabación se realiza utilizando la función **`cudaEventRecord()`**:

```
cudaEventRecord(cudaEvent_t marca, cudaStream_t stream);
```

donde `marca` es alguna de las variables declaradas anteriormente para almacenar las marcas temporales de inicio o fin y *`stream`* es la corriente o lista de tareas en la que grabar el evento, donde habitualmente se pone un `0` para tener en cuenta todas las corrientes (si es que hay más de una). A continuación lanzamos el trabajo que queremos temporizar sobre la GPU y después grabamos sobre la marca final el tiempo transcurrido.

Lo siguiente sería calcular el tiempo transcurrido entre dos eventos. Sin embargo, es necesario añadir llamada adicional después del último **`cudaEventRecord()`** y que tiene como objetivo sincronizar el *host* con el *device*. La función requerida para ello es:

```
cudaEventSynchronize(cudaEvent_t marca);
```

Esta función detiene la CPU hasta la finalización de todo el trabajo pendiente en la GPU y que precede a la llamada más reciente a **`cudaEventRecord()`**. Como ya hemos dicho, esto es necesario debido al carácter asíncrono de las funciones de CUDA. Es decir, una llamada a la función **`cudaEventRecord()`** no implica que se grabe el tiempo sino que queda pendiente

en las tareas de la GPU grabar ese tiempo. Por eso, hasta que no utilizamos la función **cudaEventSynchronize()** no podemos estar seguros de obtener la marca de tiempo correcta.

Tras la llamada a esta función sabemos que todo el trabajo anterior se habrá completado y por lo tanto también será seguro leer su correspondiente marca de tiempo. El tiempo transcurrido entre ambos eventos lo podemos calcular utilizando la función **cudaEventElapsedTime()**:

```
cudaEventElapsedTime(float *tiempo, cudaEvent_t marca1, cudaEvent_t marca2);
```

Esta función nos devuelve en la variable *tiempo*, de tipo `float`, el tiempo en milisegundos transcurrido entre los eventos *marca1* y *marca2*. Cabe mencionar que dado que los eventos se implementan directamente sobre la GPU, sólo sirven para temporizar código para la GPU y nunca para la CPU.

Para terminar podemos decir que los eventos creados también pueden destruirse utilizando una función denominada **cudaEventDestroy()**:

```
cudaEventDestroy(cudaEvent_t marca);
```

y de este modo liberamos los recursos asociados con el evento *marca*.

5. Ejemplo

En el siguiente fragmento de código se muestra cómo utilizar las funciones relacionadas con la gestión de eventos para medir el tiempo de ejecución transcurrido en la GPU. Es importante resaltar que estas funciones sólo sirven para medir tiempo de GPU, nunca para medir el tiempo de CPU (si queremos medir el tiempo de ejecución de un programa ejecutado en la CPU debemos recurrir a funciones de la biblioteca de C):

```
// MAIN: rutina principal ejecutada en el host
int main()
{
// ...

// declaracion de eventos
      cudaEvent_t start;
      cudaEvent_t stop;

// creacion de eventos
      cudaEventCreate(&start);
      cudaEventCreate(&stop);

// ...

// marca de inicio
      cudaEventRecord(start,0);

//
// codigo a temporizar en el device (kernel)
//

// marca de final
      cudaEventRecord(stop,0);

// ...

// sincronizacion GPU-CPU
      cudaEventSynchronize(stop);

// calculo del tiempo en milisegundos
      float elapsedTime;
      cudaEventElapsedTime(&elapsedTime,start,stop);

// impresion de resultados
      printf("> Tiempo de ejecucion: %f ms\n",elapsedTime);

// liberacion de recursos
      cudaEventDestroy(start);
      cudaEventDestroy(stop);

// ...

      return 0;
}
```